INSTITUTO POLITÉCNICO DO CÁVADO E DO AVE

ESCOLA SUPERIOR DE TECNOLOGIA

Licenciatura em Engenharia de Sistemas Informáticos
Unidade Curricular de Integração de Sistemas de Informação
2026/27

Trabalho Prático I

ISI Store — Loja E-commerce

Processo *Extract, Transform and Load* (ETL)

Autor: Tiago Fontes
Número: 33222
Email: a33222@alunos.ipca.pt
Repositório Git: https://github.com/Fontes9/Trabalho-Pr-tico-I---ISI.git

20 de outubro de 2026

Resumo

O presente relatório documenta o desenvolvimento de um processo de *Extract, Transform and Load* (ETL) para a loja fictícia ISI Store. O problema consistiu na dispersão de dados por sistemas isolados: vendas em CSV, clientes em JSON, catálogo em XML, *stock* em base de dados relacional PostgreSQL e catálogo externo via API REST. No estado da arte compararam-se as plataformas Pentaho Data Integration, Talend Open Studio e KNIME, tendo-se selecionado o Pentaho pela orquestração visual de transformações e *jobs*. A abordagem seguiu uma metodologia de *Design Science Research*, com especificação de requisitos, conceção da arquitetura, implementação incremental e avaliação por contagens e *previews*. Implementou-se extração multiformato, limpeza com expressões regulares, anonimização, junções e agregações, carga para *Data Mart*, exportações CSV/JSON, notificações por correio eletrónico, transferência FTP e um painel complementar em Node-RED com MQTT. Obteve-se uma execução integral válida: 13 vendas reduziram-se a 10 limpas, 6 clientes a 2 válidos, 158 produtos integrados, 10 linhas carregadas em `dw_vendas` e agregação por cidade (Braga 2077.47/3, Porto 949.98/2 e sem cliente 436.17/5). A contribuição principal é um processo reprodutível que combina integração *batch*, qualidade de dados, anonimização e visualização quase em tempo real, cumprindo as três fases obrigatórias do enunciado.

Palavras-chave: ETL; Pentaho; Node-RED; MQTT; PostgreSQL; Expressões Regulares; Qualidade de Dados.

Siglas

No presente documento utilizam-se as siglas abaixo, ordenadas alfabeticamente.

- API – *Application Programming Interface*
- BD – Base de Dados
- CSV – *Comma-Separated Values*
- ETL – *Extract, Transform and Load*
- FTP – *File Transfer Protocol*
- JSON – *JavaScript Object Notation*
- MQTT – *Message Queuing Telemetry Transport*
- REST – *Representational State Transfer*
- RGPD – Regulamento Geral sobre a Proteção de Dados
- SQL – *Structured Query Language*
- XML – *eXtensible Markup Language*

Glossário

No presente documento, os termos seguintes são utilizados com o significado indicado.

Anonimização Processo de transformação de dados pessoais de modo a impedir a identificação dos titulares, preservando a utilidade analítica dos registos.

Data Mart Subconjunto de um *data warehouse*, orientado a uma área de negócio, neste caso as vendas da ISI Store.

ETL Processo de extração, transformação e carga de dados entre sistemas, utilizado para consolidar fontes heterogéneas.

Qualidade de dados Grau de adequação dos dados ao uso pretendido, avaliado por dimensões como completude, consistência, validade e unicidade.

Streaming Processamento ou transporte contínuo de eventos, neste caso vendas publicadas em MQTT para atualização do painel.

Índice

Siglas **2**

Glossário **3**

1 Introdução **8**
1.1 Motivação . . . 8
1.2 Enquadramento . . . 8
1.3 Objetivos . . . 8
1.4 Metodologia de trabalho . . . 8
1.5 Plano de trabalho . . . 9
1.6 Estrutura do documento . . . 9

2 Estado da Arte **10**
2.1 Conceitos e fundamentos . . . 10
2.2 Soluções existentes . . . 10

3 Análise e Especificação **12**
3.1 Requisitos . . . 12
3.2 Arquitetura lógica e funcional . . . 12
3.3 Modelação de dados . . . 13
3.4 Interação e integração de serviços . . . 14

4 Implementação **15**
4.1 E_extracao.ktr . . . 15
4.2 T_limpeza.ktr . . . 18
4.3 T_clientes.ktr . . . 19
4.4 T_joins.ktr . . . 21
4.5 T_conversao.ktr . . . 22
4.6 J_principal.kjb . . . 22
4.7 Persistência e exportação . . . 22
4.8 Node-RED e painel . . . 26

5 Análise e Discussão de Resultados **28**
5.1 Validação da execução . . . 28
5.2 Discussão dos resultados . . . 28
5.2.1 Qualidade e integridade dos dados . . . 28
5.2.2 Integração e orquestração . . . 28
5.2.3 Limitações e ameaças à validade . . . 29
5.3 Vídeo de demonstração . . . 29

6 Conclusão **30**
6.1 Síntese e contribuições . . . 30
6.2 Limitações . . . 30
6.3 Trabalhos futuros . . . 30

A Contagens de validação **31**

Lista de Figuras

1 Arquitetura lógica do processo de integração da ISI Store. 13
2 Fase E no Spoon – extrações paralelas: CSV, JSON, XML, Generate+REST e Table Input. 15
3 Registo de execução E_extracao – 13 vendas, 6 clientes, 158 produtos, 1 REST e 158 stock. 16
4 Pré-visualização de vendas.csv – 13 linhas com quantidade 0, duplicado V002, `id_cliente` nulo e data errada. 16
5 Pré-visualização de clientes.json – 6 linhas com correio inválido, `email` nulo, NIF ABC123 e duplicado. 16
6 Pré-visualização de produtos.xml – 158 produtos (P001–P158) com *Loop XPath* /produtos/produto. 17
7 Pré-visualização do REST client – 1 linha com estado 200 e JSON da FakeStore (*trigger* Generate Rows dummy=1). 17
8 Pré-visualização do Table Input – 158 linhas da tabela stock. 18
9 Filter Rows vendas – 10 linhas após remover quantidade 0 e id_cliente null. 19
10 Calculator final vendas – 10 linhas com total_linha correto (V001 1799.98). 19
11 Substituição de telemóvel – normalização com expressão regular [\s-] (912 345 678 para 912345678). 20
12 Validação com expressões regulares – 6 clientes com `email_ok`, `telemovel_ok` e `nif_ok` (2 limpos C001/C003). 20
13 Clientes limpos – 2 válidos após filtro Y/Y/Y com esquema remapeado. 20
14 Anonimização RGPD – `nif_anon` ******789 e `email_anon` a***@gmail. 20
15 Junção 1 – Stream Lookup vendas+clientes, 10 linhas, nulos por integridade referencial. 21
16 Junção final – Stream Lookup 2 + produtos, 10 linhas com `nome_prod` e `categoria_prod`. 21
17 Diagrama T_join completo – vendas+clientes+produtos, 2 lookups, Sort+Group. 21
18 Group By `cidade_cliente` – 3 linhas: sem cliente 436.17/5, Braga 2077.47/3 e Porto 949.98/2. 21
19 Conversão XML para JSON – 158 produtos exportados para `produtos.json`. 22
20 *Job* principal – E, T, L, conversão, FTP CSV/JSON, Mail e Write to Log com sucesso. . 22
21 Correio eletrónico de alerta com totais por cidade. 22
22 Tabelas `dw_vendas` e `stock` na base de dados `isi_store`. 23
23 Conteúdo da tabela `stock`. 24
24 Tabela `dw_vendas` vazia antes da carga. 24
25 Ligação PostgreSQL `isi_store` testada com sucesso. 25
26 Tabela `dw_vendas` com 10 linhas e totais corretos (V001 1799.98). 26
27 Ficheiros exportados em `Dados_Saida`. 26
28 Fluxos Node-RED – *streaming* isi/vendas, agregação e painel. 27
29 Painel calculado a partir de `vendas_limpas.csv`: indicadores ETL (total 3463.62), receita e ticket médio por cidade, principais produtos, tabela por cidade e *streaming* MQTT. 27

Lista de Tabelas

1 Plano de trabalho inicial e plano final ajustado. 9
2 Quadro comparativo de plataformas de integração. 10
3 Requisitos funcionais e não funcionais. 12
4 Dicionário de dados da tabela dw_vendas. 13
5 Interfaces de interação e integração de serviços. 14
6 Fase E – diversidade de formatos e fontes. 15
7 Expressões regulares aplicadas. 20
8 Síntese das contagens e evidências da execução. 28

Índice de Códigos

1 Criação das tabelas dw_vendas e stock na base de dados isi_store. 13

1 Introdução

O presente capítulo apresenta a motivação, o enquadramento, os objetivos, a metodologia, o plano de trabalho e a estrutura do documento. A Secção 1.1 explora o problema e a ausência de uma solução integrada; a Secção 1.2 identifica o contexto curricular; a Secção 1.3 define os objetivos; a Secção 1.4 descreve a metodologia; a Secção 1.5 apresenta o plano de trabalho; e a Secção 1.6 resume a organização do relatório.

1.1 Motivação

A ISI Store mantinha sistemas isolados: vendas da loja física em CSV, registo de clientes em JSON, catálogo em XML e *stock* em PostgreSQL, sem uma visão consolidada do negócio. Esta dispersão impedia a análise rápida de receita, ticket médio e desempenho por cidade, ao mesmo tempo que os dados apresentavam problemas de qualidade: duplicado V002, quantidade zero (V005), cliente nulo (V007), data com formato incorreto (V009 com `/`), correio eletrónico inválido e NIF não numérico.

A ausência de um processo de integração obrigava a reconciliações manuais e aumentava o risco de decisões baseadas em informação incompleta. O contexto atual favorece a resolução deste problema: as plataformas ETL visuais permitem orquestrar fontes heterogéneas, os *dashboards* facilitam a exploração de indicadores e os protocolos de mensagens, como MQTT, possibilitam atualizações quase em tempo real. O trabalho parte, assim, de um problema concreto de integração e qualidade de dados, alinhado com o enunciado da unidade curricular [**?**].

1.2 Enquadramento

O trabalho foi desenvolvido individualmente no âmbito da Unidade Curricular de Integração de Sistemas de Informação (ISI), da Licenciatura em Engenharia de Sistemas Informáticos, no ano letivo 2026/27. Explorou-se o tema ISI Store, uma loja fictícia de informática e mobiliário de escritório.

A plataforma principal foi o Pentaho Data Integration (Kettle/Spoon) *Community Edition* 10.2. Como ferramentas complementares utilizaram-se PostgreSQL com DBeaver, Node-RED com Mosquitto e o painel Node-RED Dashboard. O repositório Git do projeto está disponível em `https://github.com/Fontes9/Trabalho-Pr-tico-I---ISI.git`.

1.3 Objetivos

Definiram-se quatro objetivos, intencionalmente poucos e claros:

1. analisar as fontes heterogéneas e especificar os requisitos de integração;
2. implementar um processo ETL completo, com extração, limpeza, validação, junções, agregações e carga;
3. integrar a orquestração, as notificações, a transferência de ficheiros e a visualização de indicadores;
4. avaliar os resultados e documentar o processo de forma reprodutível.

Os objetivos enquadram-se no enunciado da unidade curricular [**?**] e orientam a análise apresentada nos capítulos seguintes.

1.4 Metodologia de trabalho

Adotou-se uma metodologia de investigação aplicada inspirada em *Design Science Research* (DSR) [**?**]. O processo organizou-se em seis atividades: identificação do problema; revisão de literatura e comparação de soluções; especificação de requisitos e da arquitetura; implementação do artefacto; avaliação por contagens, *previews* e execução integral; e comunicação dos resultados.

A componente de investigação incidiu na comparação de plataformas, na definição de regras de qualidade e na avaliação crítica dos resultados. Assim, o trabalho não se limitou ao desenvolvimento de software: incluiu revisão de literatura, especificação, implementação e discussão fundamentada, distinguindo-se de uma abordagem meramente técnica.

1.5 Plano de trabalho

O plano inicial foi organizado em semanas relativas ao início do projeto, com revisão semanal e ajuste progressivo das metas. A Tabela 1 sintetiza o plano inicial e o plano final ajustado.

Fase	Plano inicial	Plano final
Revisão de literatura e requisitos	Semanas 1–2	Semanas 1–2
Conceção da arquitetura e modelo de dados	Semana 3	Semanas 3–4
Implementação E–T–L	Semanas 4–6	Semanas 4–6
Orquestração, FTP e correio eletrónico	Semana 7	Semana 7
Painel e MQTT	Semana 8	Semanas 8–9
Validação, redação e revisão	Semanas 9–10	Semanas 10–11

Tabela 1: Plano de trabalho inicial e plano final ajustado.

As semanas indicadas são relativas ao início do projeto, e o plano final foi ajustado à disponibilidade das fontes e do ambiente de laboratório.

1.6 Estrutura do documento

A Secção 2 apresenta o estado da arte e a comparação de soluções existentes. A Secção 3 descreve a análise e especificação, incluindo requisitos, arquitetura, modelo de dados e interfaces. A Secção 4 documenta a implementação das transformações, do *Job* e do painel. A Secção 5 analisa e discute os resultados. A Secção 6 apresenta a conclusão, as limitações e os trabalhos futuros. Por fim, constam as referências bibliográficas e o anexo com as contagens de validação.

2 Estado da Arte

O presente capítulo enquadra teoricamente o trabalho e identifica soluções existentes. A Secção 2.1 apresenta os conceitos fundamentais de ETL, qualidade de dados, *Data Mart*, orquestração e privacidade. A Secção 2.2 compara plataformas de integração e justifica a escolha do Pentaho.

2.1 Conceitos e fundamentos

O ETL é o processo responsável por extrair dados de fontes heterogéneas, transformá-los de acordo com regras de negócio e carregá-los num destino consolidado [**?**]. A fase de extração lida com formatos e protocolos distintos; a transformação concentra a limpeza, a validação, a normalização e o enriquecimento; a carga persiste os dados em estruturas orientadas à análise, como um *Data Mart*.

A qualidade de dados é avaliada por dimensões como completude, consistência, validade e unicidade. No caso da ISI Store, estas dimensões traduzem-se na remoção de duplicados, na rejeição de quantidades nulas, na validação de correio eletrónico e NIF e na correção de datas. A qualidade não é apenas um requisito técnico: condiciona a confiança nos indicadores e a utilidade do processo para a decisão.

Um *Data Mart* é um subconjunto de um *data warehouse*, orientado a uma área de negócio e ajustado às necessidades de análise [**?**]. A orquestração em *jobs* permite encadear transformações, tratar erros e executar tarefas complementares, como notificações e transferências de ficheiros. Em paralelo, a integração de dados pessoais exige conformidade com o RGPD, designadamente minimização e anonimização [**?**].

A integração pode ser *batch* ou em *streaming*. O processo principal deste trabalho é *batch*, porque o *Job* é executado sobre ficheiros e uma base de dados relacional PostgreSQL [**?**]. A visualização complementar recorre a MQTT para publicar vendas e atualizar um painel, aproximando a análise do tempo real [**?**, **?**].

2.2 Soluções existentes

Identificaram-se três soluções no domínio da integração de dados: Pentaho Data Integration, Talend Open Studio e KNIME. A Tabela 2 compara-as segundo critérios alinhados com os objetivos do trabalho: orquestração visual, diversidade de conectores, notificações e FTP, curva de aprendizagem, licenciamento e integração com *streaming*.

Critério	Pentaho 10.2	Talend	KNIME
Orquestração visual ETL + *Job*	Sim	Parcial	Parcial
Conectores CSV/JSON/XML/REST/BD	Sim	Sim	Sim
Correio eletrónico e FTP no *Job*	Sim	Parcial	Não
Curva de aprendizagem	Média	Média-alta	Média
Licenciamento	*Community Edition*	*Open Studio*	*Open source*
Integração com MQTT/*streaming*	Via plugin/Node-RED	Via extensão	Via extensão
Foco principal	Integração e orquestração	Integração	Análise de dados

Tabela 2: Quadro comparativo de plataformas de integração.

A Tabela 2 mostra que as três plataformas cobrem os conectores necessários, mas diferem na orquestração e nas tarefas complementares. O Pentaho adequou-se melhor porque combina transformações e *jobs* com correio eletrónico e FTP na mesma ferramenta visual, enquanto o Talend exige maior configuração e o KNIME se orienta mais para análise de dados [**?**, **?**, **?**].

A escolha do Pentaho foi também influenciada pela existência de uma edição comunitária e pela possibilidade de integrar o Node-RED como ferramenta complementar de visualização e MQTT. Esta combinação permite separar a integração *batch* da apresentação quase em tempo real, sem substituir a plataforma principal.

3 Análise e Especificação

O presente capítulo descreve a análise do problema e a especificação da solução. A Secção 3.1 apresenta os requisitos; a Secção 3.2 a arquitetura lógica e funcional; a Secção 3.3 o modelo de dados; e a Secção 3.4 as interfaces de interação e integração de serviços.

3.1 Requisitos

Os requisitos foram derivados do enunciado e da análise das fontes de dados [**?**]. A Tabela 3 sistematiza os requisitos funcionais (RF) e não funcionais (RNF), com a respetiva prioridade.

ID	Requisito	Tipo	Prioridade
RF01	Extrair vendas de CSV, clientes de JSON, produtos de XML, *stock* de PostgreSQL e catálogo de API REST	Funcional	Alta
RF02	Limpar e validar vendas: duplicados, quantidade, cliente, data e total de linha	Funcional	Alta
RF03	Validar e anonimizar clientes, incluindo correio eletrónico, telemóvel e NIF	Funcional	Alta
RF04	Juntar vendas, clientes e produtos e agregar por cidade	Funcional	Alta
RF05	Carregar o resultado no *Data Mart* `dw_vendas`	Funcional	Alta
RF06	Exportar `vendas_limpas.csv` e `resumo.json`	Funcional	Média
RF07	Enviar correio eletrónico com os totais por cidade	Funcional	Média
RF08	Transferir os ficheiros exportados por FTP	Funcional	Média
RF09	Publicar vendas em MQTT e apresentar painel de indicadores	Funcional	Média
RF10	Registar a execução em *log*	Funcional	Média
RNF01	Garantir conformidade com o RGPD na anonimização de dados pessoais	Não funcional	Alta
RNF02	Assegurar rastreabilidade e reprodutibilidade da execução	Não funcional	Alta
RNF03	Proporcionar uma visualização simples e legível dos indicadores	Não funcional	Média
RNF04	Concluir a execução do *Job* sem intervenção manual	Não funcional	Média
RNF05	Permitir a portabilidade dos artefactos entre ambientes locais	Não funcional	Média

Tabela 3: Requisitos funcionais e não funcionais.

Os requisitos RF01 a RF10 cobrem o percurso completo dos dados, desde as fontes até ao consumo. Os requisitos RNF01 e RNF02 são particularmente relevantes porque o processo manipula dados pessoais e tem de ser auditável.

3.2 Arquitetura lógica e funcional

A arquitetura adotada segue uma organização em camadas: fontes, integração, persistência e consumo. A Figura 1 representa esta organização de forma lógica, sem impor uma infraestrutura física única.

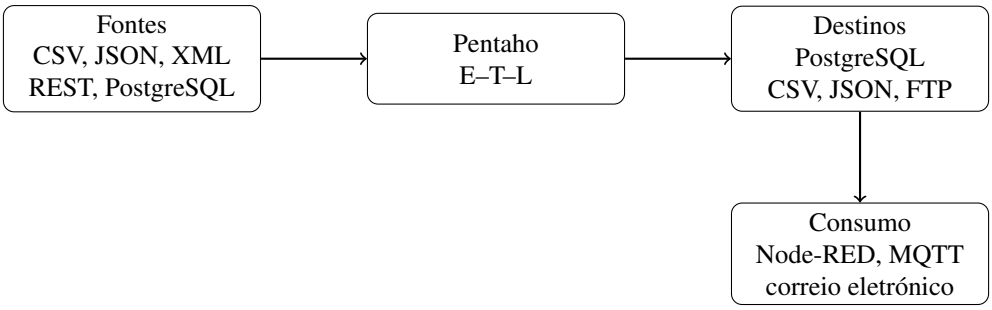


Figura 1: Arquitetura lógica do processo de integração da ISI Store.

A camada de fontes reúne os sistemas isolados da loja. A camada de integração concentra as transformações e o *Job* do Pentaho. A camada de persistência inclui o *Data Mart*, os ficheiros exportados e a pasta que simula o servidor FTP. A camada de consumo integra o correio eletrónico, o painel Node-RED e o tópico MQTT.

Esta separação permite substituir uma fonte ou um destino sem alterar todo o processo. Permite também distinguir a integração *batch*, responsável pela consolidação, da visualização quase em tempo real, responsável pela apresentação dos indicadores.

3.3 Modelação de dados

O modelo de dados inclui a tabela de factos `dw_vendas` e a tabela de apoio `stock`. O Código 1 apresenta a criação de ambas na base de dados `isi_store`, e a Tabela 4 descreve os campos de `dw_vendas`.

```
CREATE TABLE dw_vendas (
  id_venda VARCHAR(10) PRIMARY KEY,
  id_cliente VARCHAR(10),
  id_produto VARCHAR(10),
  data DATE,
  quantidade INT,
  preco_unit NUMERIC(10,2),
  total_linha NUMERIC(10,2)
);
CREATE TABLE stock (
  id_produto VARCHAR(10) PRIMARY KEY,
  stock_atual INT
);
```

Código 1: Criação das tabelas dw_vendas e stock na base de dados isi_store.

Campo	Tipo	Descrição
`id_venda`	VARCHAR(10)	Identificador da venda (chave primária).
`id_cliente`	VARCHAR(10)	Identificador do cliente; pode ser nulo após a limpeza.
`id_produto`	VARCHAR(10)	Identificador do produto vendido.
`data`	DATE	Data da venda, normalizada para o formato ISO.
`quantidade`	INT	Número de unidades vendidas; deve ser superior a zero.
`preco_unit`	NUMERIC(10,2)	Preço unitário do produto.
`total_linha`	NUMERIC(10,2)	Total da linha, calculado como quantidade × preço unitário.

Tabela 4: Dicionário de dados da tabela dw_vendas.

O Código 1 e a Tabela 4 mostram que o modelo é simples e orientado à análise. A tabela `stock`

mantém a chave do produto e a quantidade disponível, permitindo cruzar vendas e existências sem alterar a granularidade da tabela de factos.

3.4 Interação e integração de serviços

A solução integra ficheiros, base de dados, correio eletrónico, FTP e MQTT. A Tabela 5 resume as interfaces e a respetiva finalidade.

Interface	Tecnologia	Direção	Finalidade
Vendas	CSV	Entrada	Leitura das vendas da loja física.
Clientes	JSON	Entrada	Leitura dos dados de clientes.
Produtos	XML	Entrada	Leitura do catálogo de produtos.
Catálogo externo	REST/JSON	Entrada	Recurso de apoio à integração.
Stock	PostgreSQL/JDBC	Entrada	Leitura das existências.
Data Mart	PostgreSQL/JDBC	Saída	Persistência das vendas limpas.
Alerta	SMTP	Saída	Envio dos totais por cidade.
Ficheiros	FTP	Saída	Transferência de CSV e JSON.
Painel	HTTP e MQTT	Saída/Entrada	Visualização e atualização do painel.

Tabela 5: Interfaces de interação e integração de serviços.

A Tabela 5 evidencia que a solução não se limita a mover dados: define também pontos de consumo e de notificação. O painel Node-RED lê o CSV consolidado e recebe eventos MQTT, enquanto o correio eletrónico e o FTP comunicam o resultado da execução.

4 Implementação

O presente capítulo descreve a implementação da solução. A Secção 4.1 apresenta a extração; a Secção 4.2 a limpeza de vendas; a Secção 4.3 a validação e anonimização de clientes; a Secção 4.4 as junções e agregações; a Secção 4.5 a conversão XML–JSON; a Secção 4.6 a orquestração; a Secção 4.7 a persistência e exportação; e a Secção 4.8 o painel Node-RED.

4.1 E_extracao.ktr

Implementou-se a leitura paralela das fontes antes do processamento principal, com *schemas* heterogéneos preservados; a junção efetuou-se na fase de transformação. A Tabela 6 resume as fontes e os *steps* utilizados, a Figura 2 apresenta o diagrama com os seis *steps* paralelos e a Figura 3 o log de execução: vendas W=13, clientes W=6, produtos W=158, REST W=1 e *stock* W=158.

Fonte	Formato	*Step* Pentaho
vendas.csv (ficheiro local)	CSV	CSV File Input
clientes.json (ficheiro local)	JSON	JSON Input
produtos.xml (ficheiro local)	XML	Get Data from XML
API FakeStore	JSON REST	Generate Rows + REST Client
stock BD PostgreSQL	Tabela SQL	Table Input

Tabela 6: Fase E – diversidade de formatos e fontes.

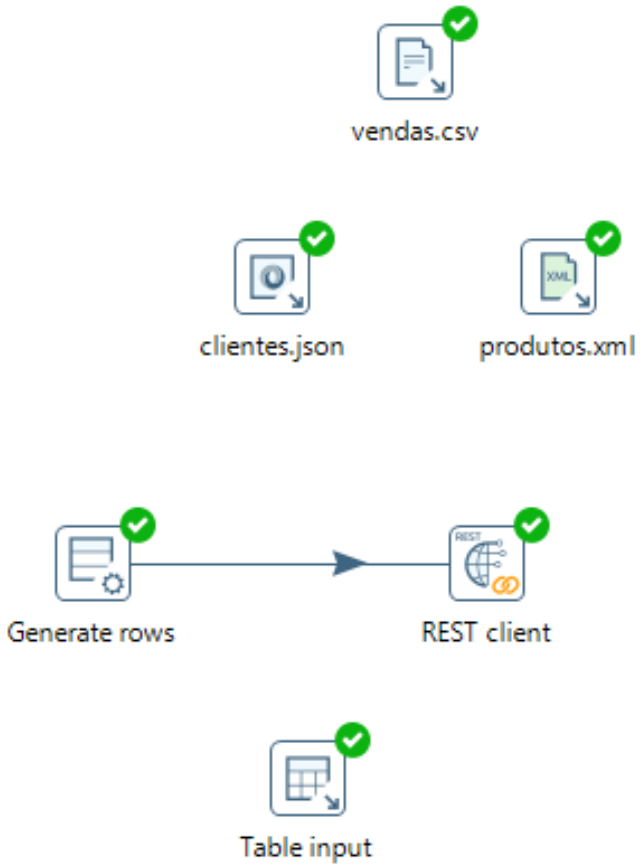


Figura 2: Fase E no Spoon – extrações paralelas: CSV, JSON, XML, Generate+REST e Table Input.

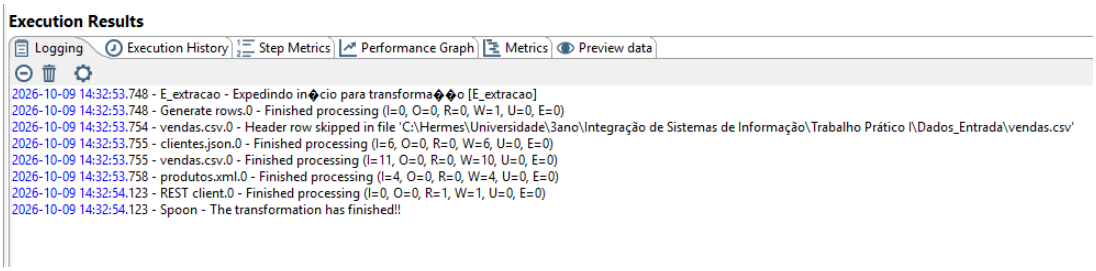


Figura 3: Registo de execução E_extracao – 13 vendas, 6 clientes, 158 produtos, 1 REST e 158 stock.

Os dados de entrada evidenciam os problemas a tratar: a Figura 4 mostra as 13 vendas com quantidade zero, duplicado V002, cliente nulo e data errada; a Figura 5 mostra os 6 clientes com correio inválido, correio nulo, NIF não numérico e duplicado; a Figura 6 mostra os 158 produtos com *Loop XPath*; a Figura 7 mostra a resposta 200 da API FakeStore obtida com *trigger* Generate Rows [**?**]; e a Figura 8 mostra as 158 linhas de *stock*.

Examine preview data

Rows of step: vendas.csv (13 rows)

#	id_venda	id_cliente	id_produto	data	quantidade	preco_unit
1	V001	C001	P001	2026-10-01	2	899.99
2	V002	C002	P002	2026-10-02	1	29.9
3	V003	C003	P001	2026-10-03	1	899.99
4	V004	C002	P003	2026-10-04	3	15.5
5	V005	C004	P002	2026-10-05	0	29.9
6	V006	C001	P003	2026-10-06	5	15.5
7	V002	C002	P002	2026-10-02	1	29.9
8	V007	<null>	P001	2026-10-07	1	899.99
9	V008	C005	P004	2026-10-08	2	49.99
10	V009	C003	P004	2026/10/09	1	49.99
11	V010	C001	P005	2026-10-10	1	199.99
12	V011	C002	P006	2026-10-11	2	79.9
13	V012	C004	P007	2026-10-12	1	99.99

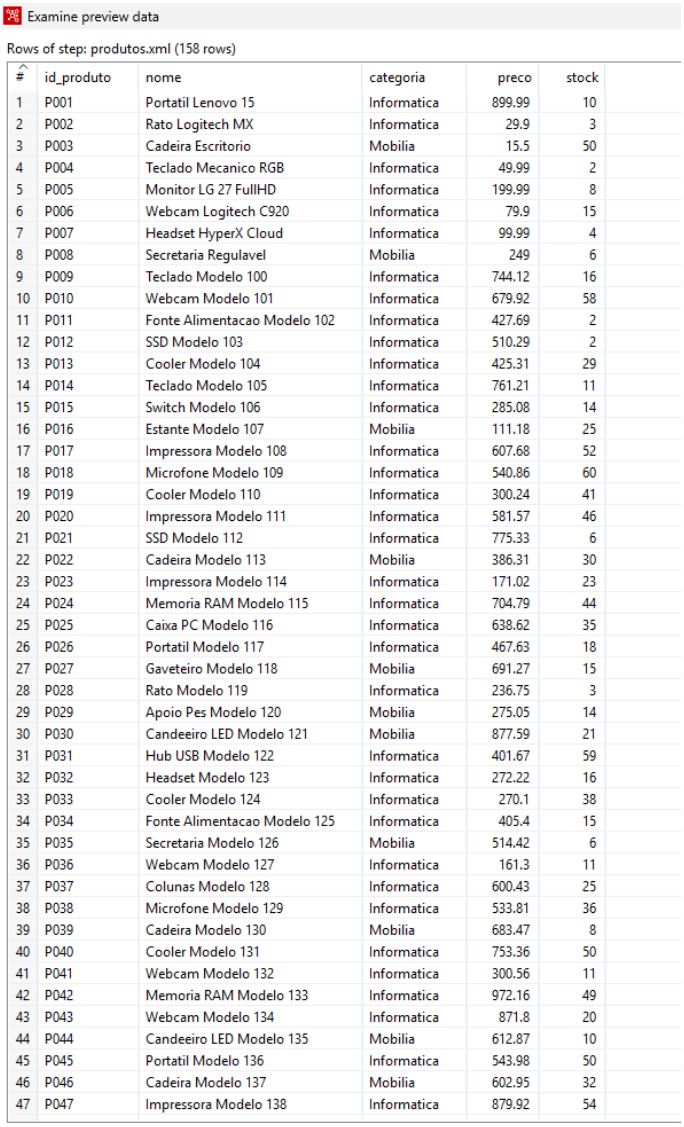
Figura 4: Pré-visualização de vendas.csv – 13 linhas com quantidade 0, duplicado V002, `id_cliente` nulo e data errada.

Examine preview data

Rows of step: clientes.json (6 rows)

#	id_cliente	email	nome	telemovel	nif	cidade
1	C001	ana.silva@gmail.com	ana silva	912 345 678	123456789	Braga
2	C002	joao.pereira@mail	JOAO PEREIRA	913-456-789	987654321	Guimaraes
3	C003	maria.costa@hotmail.com	Maria Costa	914456789	456789123	Porto
4	C004	<null>	Pedro Santos	915 456 789	321654987	Braga
5	C005	ines.rocha@gmail.com	Ines Rocha	91645678	ABC123	Lisboa
6	C002	joao.pereira@mail	JOAO PEREIRA	913-456-789	987654321	Guimaraes

Figura 5: Pré-visualização de clientes.json – 6 linhas com correio inválido, `email` nulo, NIF ABC123 e duplicado.


Examine preview data

Rows of step: produtos.xml (158 rows)

#	id_produto	nome	categoria	preco	stock
1	P001	Portatil Lenovo 15	Informatica	899.99	10
2	P002	Rato Logitech MX	Informatica	29.9	3
3	P003	Cadeira Escritorio	Mobilia	15.5	50
4	P004	Teclado Mecanico RGB	Informatica	49.99	2
5	P005	Monitor LG 27 FullHD	Informatica	199.99	8
6	P006	Webcam Logitech C920	Informatica	79.9	15
7	P007	Headset HyperX Cloud	Informatica	99.99	4
8	P008	Secretaria Regulavel	Mobilia	249	6
9	P009	Teclado Modelo 100	Informatica	744.12	16
10	P010	Webcam Modelo 101	Informatica	679.92	58
11	P011	Fonte Alimentacao Modelo 102	Informatica	427.69	2
12	P012	SSD Modelo 103	Informatica	510.29	2
13	P013	Cooler Modelo 104	Informatica	425.31	29
14	P014	Teclado Modelo 105	Informatica	761.21	11
15	P015	Switch Modelo 106	Informatica	285.08	14
16	P016	Estante Modelo 107	Mobilia	111.18	25
17	P017	Impressora Modelo 108	Informatica	607.68	52
18	P018	Microfone Modelo 109	Informatica	540.86	60
19	P019	Cooler Modelo 110	Informatica	300.24	41
20	P020	Impressora Modelo 111	Informatica	581.57	46
21	P021	SSD Modelo 112	Informatica	775.33	6
22	P022	Cadeira Modelo 113	Mobilia	386.31	30
23	P023	Impressora Modelo 114	Informatica	171.02	23
24	P024	Memoria RAM Modelo 115	Informatica	704.79	44
25	P025	Caixa PC Modelo 116	Informatica	638.62	35
26	P026	Portatil Modelo 117	Informatica	467.63	18
27	P027	Gaveteiro Modelo 118	Mobilia	691.27	15
28	P028	Rato Modelo 119	Informatica	236.75	3
29	P029	Apoio Pes Modelo 120	Mobilia	275.05	14
30	P030	Candeeiro LED Modelo 121	Mobilia	877.59	21
31	P031	Hub USB Modelo 122	Informatica	401.67	59
32	P032	Headset Modelo 123	Informatica	272.22	16
33	P033	Cooler Modelo 124	Informatica	270.1	38
34	P034	Fonte Alimentacao Modelo 125	Informatica	405.4	15
35	P035	Secretaria Modelo 126	Mobilia	514.42	6
36	P036	Webcam Modelo 127	Informatica	161.3	11
37	P037	Colunas Modelo 128	Informatica	600.43	25
38	P038	Microfone Modelo 129	Informatica	533.81	36
39	P039	Cadeira Modelo 130	Mobilia	683.47	8
40	P040	Cooler Modelo 131	Informatica	753.36	50
41	P041	Webcam Modelo 132	Informatica	300.56	11
42	P042	Memoria RAM Modelo 133	Informatica	972.16	49
43	P043	Webcam Modelo 134	Informatica	871.8	20
44	P044	Candeeiro LED Modelo 135	Mobilia	612.87	10
45	P045	Portatil Modelo 136	Informatica	543.98	50
46	P046	Cadeira Modelo 137	Mobilia	602.95	32
47	P047	Impressora Modelo 138	Informatica	879.92	54

Figura 6: Pré-visualização de produtos.xml – 158 produtos (P001–P158) com *Loop XPath* /produtos/produto.

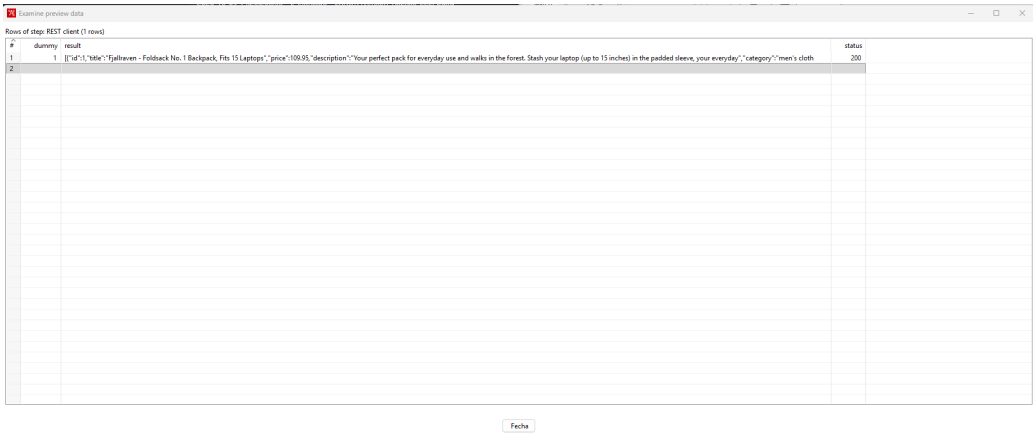
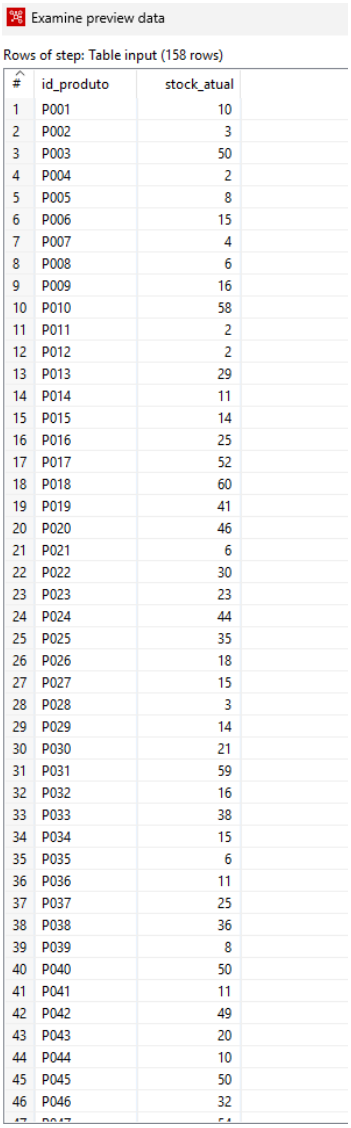


Figura 7: Pré-visualização do REST client – 1 linha com estado 200 e JSON da FakeStore (*trigger* Generate Rows dummy=1).

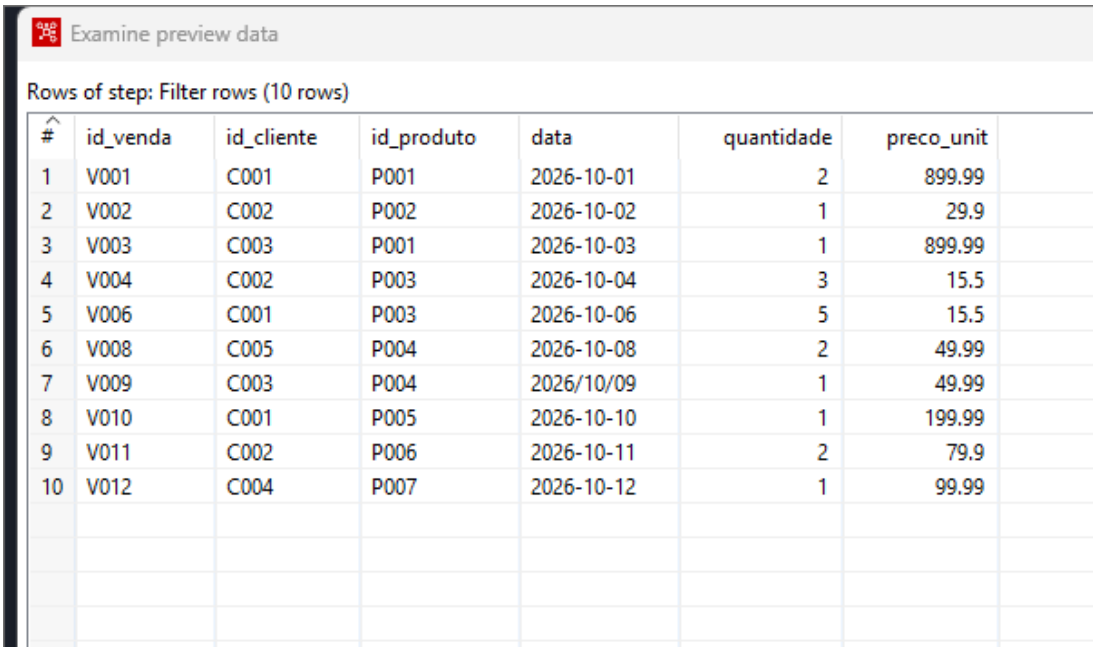


#	id_produto	stock_atual
1	P001	10
2	P002	3
3	P003	50
4	P004	2
5	P005	8
6	P006	15
7	P007	4
8	P008	6
9	P009	16
10	P010	58
11	P011	2
12	P012	2
13	P013	29
14	P014	11
15	P015	14
16	P016	25
17	P017	52
18	P018	60
19	P019	41
20	P020	46
21	P021	6
22	P022	30
23	P023	23
24	P024	44
25	P025	35
26	P026	18
27	P027	15
28	P028	3
29	P029	14
30	P030	21
31	P031	59
32	P032	16
33	P033	38
34	P034	15
35	P035	6
36	P036	11
37	P037	25
38	P038	36
39	P039	8
40	P040	50
41	P041	11
42	P042	49
43	P043	20
44	P044	10
45	P045	50
46	P046	32

Figura 8: Pré-visualização do Table Input – 158 linhas da tabela stock.

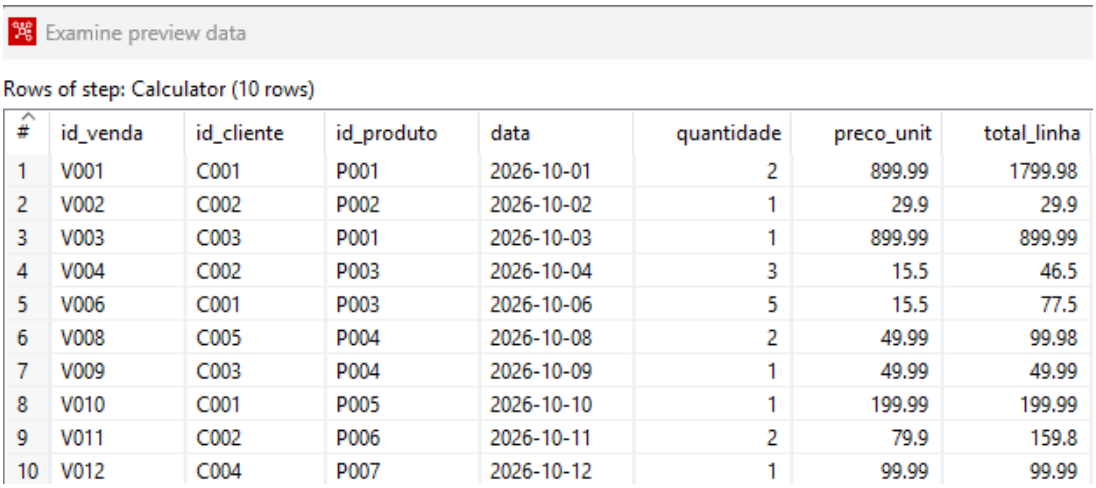
4.2 T_limpeza.ktr

Aplicou-se Sort Rows por `id_venda` seguido de Unique Rows, que removeu o duplicado V002 (13 para 12), e Filter Rows com `quantidade>0 AND id_cliente IS NOT NULL`, que removeu V005 e V007 (12 para 10), conforme a Figura 9. Efetuou-se a substituição da data de `/` para `-` (V009) e calculou-se `total_linha=quantidade*preco_unit` em Calculator BigNumber com precisão 2, após correção do *locale* (Decimal `.` sem agrupamento), conforme a Figura 10, com V001 a 1799.98.



#	id_venda	id_cliente	id_produto	data	quantidade	preco_unit
1	V001	C001	P001	2026-10-01	2	899.99
2	V002	C002	P002	2026-10-02	1	29.9
3	V003	C003	P001	2026-10-03	1	899.99
4	V004	C002	P003	2026-10-04	3	15.5
5	V006	C001	P003	2026-10-06	5	15.5
6	V008	C005	P004	2026-10-08	2	49.99
7	V009	C003	P004	2026/10/09	1	49.99
8	V010	C001	P005	2026-10-10	1	199.99
9	V011	C002	P006	2026-10-11	2	79.9
10	V012	C004	P007	2026-10-12	1	99.99

Figura 9: Filter Rows vendas – 10 linhas após remover quantidade 0 e id_cliente null.

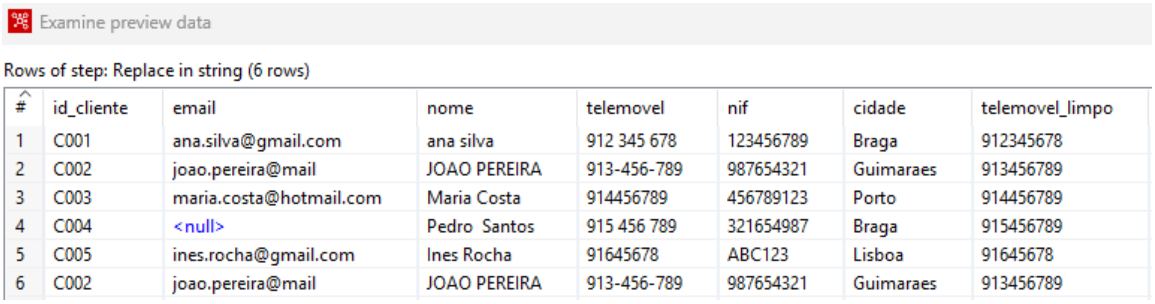


#	id_venda	id_cliente	id_produto	data	quantidade	preco_unit	total_linha
1	V001	C001	P001	2026-10-01	2	899.99	1799.98
2	V002	C002	P002	2026-10-02	1	29.9	29.9
3	V003	C003	P001	2026-10-03	1	899.99	899.99
4	V004	C002	P003	2026-10-04	3	15.5	46.5
5	V006	C001	P003	2026-10-06	5	15.5	77.5
6	V008	C005	P004	2026-10-08	2	49.99	99.98
7	V009	C003	P004	2026-10-09	1	49.99	49.99
8	V010	C001	P005	2026-10-10	1	199.99	199.99
9	V011	C002	P006	2026-10-11	2	79.9	159.8
10	V012	C004	P007	2026-10-12	1	99.99	99.99

Figura 10: Calculator final vendas – 10 linhas com total_linha correto (V001 1799.98).

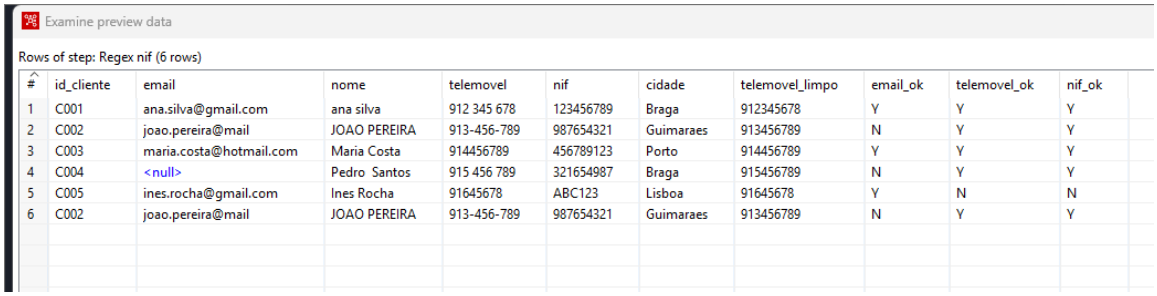
4.3 T_clientes.ktr

Normalizou-se o telemóvel com Replace por expressão regular `[\s-]` (Figura 11), validaram-se correio eletrónico, telemóvel limpo e NIF em três avaliações Regex em série (Figura 12) e filtraram-se os triplos Y/Y/Y, restando 2 clientes limpos (C001 e C003), conforme a Figura 13. Aplicou-se anonimização para conformidade com o RGPD (Figura 14) [**?**].



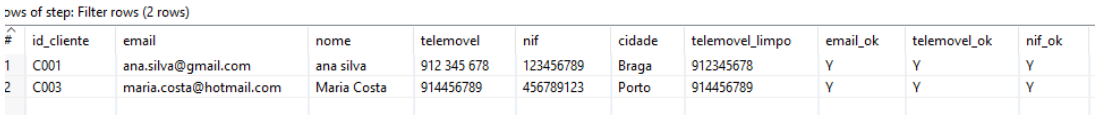
#	id_cliente	email	nome	telemovel	nif	cidade	telemovel_limpo
1	C001	ana.silva@gmail.com	ana silva	912 345 678	123456789	Braga	912345678
2	C002	joao.pereira@mail	JOAO PEREIRA	913-456-789	987654321	Guimaraes	913456789
3	C003	maria.costa@hotmail.com	Maria Costa	914456789	456789123	Porto	914456789
4	C004	<null>	Pedro Santos	915 456 789	321654987	Braga	915456789
5	C005	ines.rocha@gmail.com	Ines Rocha	91645678	ABC123	Lisboa	91645678
6	C002	joao.pereira@mail	JOAO PEREIRA	913-456-789	987654321	Guimaraes	913456789

Figura 11: Substituição de telemóvel – normalização com expressão regular [\s-] (912 345 678 para 912345678).



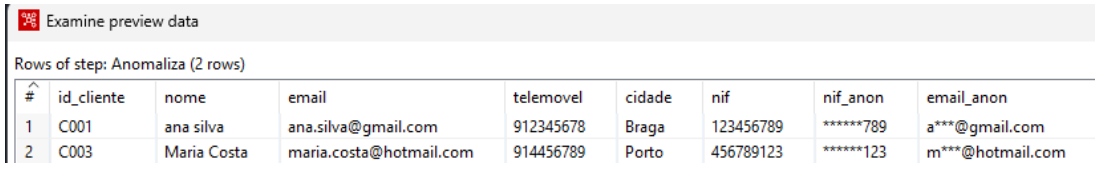
#	id_cliente	email	nome	telemovel	nif	cidade	telemovel_limpo	email_ok	telemovel_ok	nif_ok
1	C001	ana.silva@gmail.com	ana silva	912 345 678	123456789	Braga	912345678	Y	Y	Y
2	C002	joao.pereira@mail	JOAO PEREIRA	913-456-789	987654321	Guimaraes	913456789	N	Y	Y
3	C003	maria.costa@hotmail.com	Maria Costa	914456789	456789123	Porto	914456789	Y	Y	Y
4	C004	<null>	Pedro Santos	915 456 789	321654987	Braga	915456789	N	Y	Y
5	C005	ines.rocha@gmail.com	Ines Rocha	91645678	ABC123	Lisboa	91645678	Y	N	N
6	C002	joao.pereira@mail	JOAO PEREIRA	913-456-789	987654321	Guimaraes	913456789	N	Y	Y

Figura 12: Validação com expressões regulares – 6 clientes com `email_ok`, `telemovel_ok` e `nif_ok` (2 limpos C001/C003).



#	id_cliente	email	nome	telemovel	nif	cidade	telemovel_limpo	email_ok	telemovel_ok	nif_ok
1	C001	ana.silva@gmail.com	ana silva	912 345 678	123456789	Braga	912345678	Y	Y	Y
2	C003	maria.costa@hotmail.com	Maria Costa	914456789	456789123	Porto	914456789	Y	Y	Y

Figura 13: Clientes limpos – 2 válidos após filtro Y/Y/Y com esquema remapeado.



#	id_cliente	nome	email	telemovel	cidade	nif	nif_anon	email_anon
1	C001	ana silva	ana.silva@gmail.com	912345678	Braga	123456789	******789	a***@gmail.com
2	C003	Maria Costa	maria.costa@hotmail.com	914456789	Porto	456789123	******123	m***@hotmail.com

Figura 14: Anonimização RGPD – `nif_anon` ******789 e `email_anon` a***@gmail.

A Tabela 7 sistematiza as expressões regulares aplicadas na limpeza e validação dos dados.

Campo	Expressão regular
Correio eletrónico	`^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$`
Telemóvel PT após limpar `[\s-]`	`^9[1236][0-9]{7}$`
NIF	`^[0-9]{9}$`
Data	`^\d{4}-\d{2}-\d{2}$`

Tabela 7: Expressões regulares aplicadas.

4.4 T_joins.ktr

Efetuaram-se dois Stream Lookups, sem ordenação prévia, ao contrário do Merge Join: vendas com clientes por `id_cliente` (Figura 15) e o resultado com produtos por `id_produto` (Figura 16). Os nulos em C002, C004 e C005 refletem integridade referencial, pois esses clientes foram filtrados como sujos. O diagrama completo encontra-se na Figura 17 e a agregação por cidade na Figura 18: sem cliente 436.17/5, Braga 2077.47/3 e Porto 949.98/2. Sem Sort prévio obtinham-se 9 linhas.

Rows of step: Stream lookup (10 rows)

#	id_venda	id_cliente	id_produto	data	quantidade	preco_unit	total_linha	nome_cliente	cidade_cliente
1	V001	C001	P001	2026-10-01	2	899.99	1799.98	ana silva	Braga
2	V002	C002	P002	2026-10-02	1	29.9	29.9	<null>	<null>
3	V003	C003	P001	2026-10-03	1	899.99	899.99	Maria Costa	Porto
4	V004	C002	P003	2026-10-04	3	15.5	46.5	<null>	<null>
5	V006	C001	P003	2026-10-06	5	15.5	77.5	ana silva	Braga
6	V008	C005	P004	2026-10-08	2	49.99	99.98	<null>	<null>
7	V009	C003	P004	2026-10-09	1	49.99	49.99	Maria Costa	Porto
8	V010	C001	P005	2026-10-10	1	199.99	199.99	ana silva	Braga
9	V011	C002	P006	2026-10-11	2	79.9	159.8	<null>	<null>
10	V012	C004	P007	2026-10-12	1	99.99	99.99	<null>	<null>

Figura 15: Junção 1 – Stream Lookup vendas+clientes, 10 linhas, nulos por integridade referencial.

Rows of step: Stream lookup 2 (10 rows)

#	id_venda	id_cliente	id_produto	data	quantidade	preco_unit	total_linha	nome_cliente	cidade_cliente	nome_prod	categoria_prod
1	V001	C001	P001	2026-10-01	2	899.99	1799.98	ana silva	Braga	Portatil Lenovo 15	Informatica
2	V002	C002	P002	2026-10-02	1	29.9	29.9	<null>	<null>	Rato Logitech MX	Informatica
3	V003	C003	P001	2026-10-03	1	899.99	899.99	Maria Costa	Porto	Portatil Lenovo 15	Informatica
4	V004	C002	P003	2026-10-04	3	15.5	46.5	<null>	<null>	Cadeira Escritorio	Mobilia
5	V006	C001	P003	2026-10-06	5	15.5	77.5	ana silva	Braga	Cadeira Escritorio	Mobilia
6	V008	C005	P004	2026-10-08	2	49.99	99.98	<null>	<null>	Teclado Mecanico RGB	Informatica
7	V009	C003	P004	2026-10-09	1	49.99	49.99	Maria Costa	Porto	Teclado Mecanico RGB	Informatica
8	V010	C001	P005	2026-10-10	1	199.99	199.99	ana silva	Braga	Monitor LG 27 FullHD	Informatica
9	V011	C002	P006	2026-10-11	2	79.9	159.8	<null>	<null>	Webcam Logitech C920	Informatica
10	V012	C004	P007	2026-10-12	1	99.99	99.99	<null>	<null>	Headset HyperX Cloud	Informatica

Figura 16: Junção final – Stream Lookup 2 + produtos, 10 linhas com `nome_prod` e `categoria_prod`.

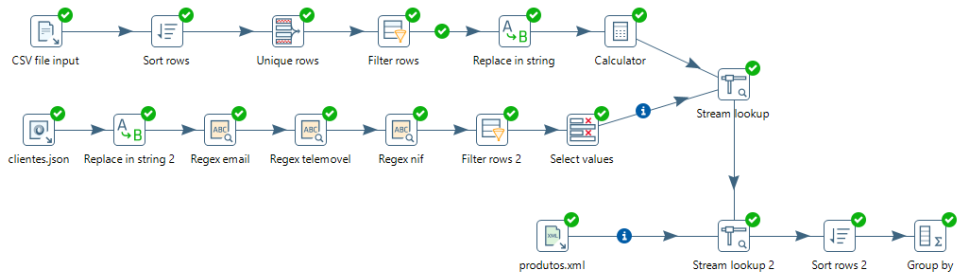


Figura 17: Diagrama T_join completo – vendas+clientes+produtos, 2 lookups, Sort+Group.

Rows of step: Group by (3 rows)

#	cidade_cliente	total_cidade	num_vendas
1	<null>	436.17	5
2	Braga	2077.47	3
3	Porto	949.98	2

Figura 18: Group By `cidade_cliente` – 3 linhas: sem cliente 436.17/5, Braga 2077.47/3 e Porto 949.98/2.

4.5 T_conversao.ktr

A transformação de conversão reutiliza o catálogo de produtos e gera `produtos.json` com 158 registos no bloco `produtos`, preservando o preço com duas casas decimais. O diagrama encontra-se na Figura 19.

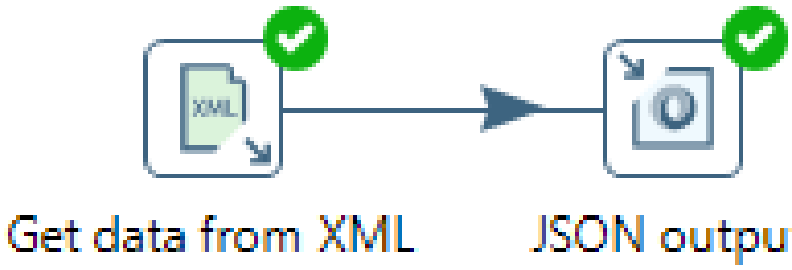


Figura 19: Conversão XML para JSON – 158 produtos exportados para `produtos.json`.

4.6 J_principal.kjb

Implementou-se o *Job* com a sequência START, E_extracao, T_joins, L_carga, T_conversao, FTP_csv, FTP_json, Mail e Write to log, conforme a Figura 20, com execução integral válida. Utilizaram-se *entries* Transformation para as quatro transformações, Send Mail para o alerta com os totais 2077.47, 949.98 e 436.17 (Figura 21), FTP Put com um *step* por ficheiro, devido à limitação da versão 10.2 a um *wildcard* por *step*, para o servidor local `127.0.0.1:21`, e Write to Log para registo de sucesso.



Figura 20: *Job* principal – E, T, L, conversão, FTP CSV/JSON, Mail e Write to Log com sucesso.

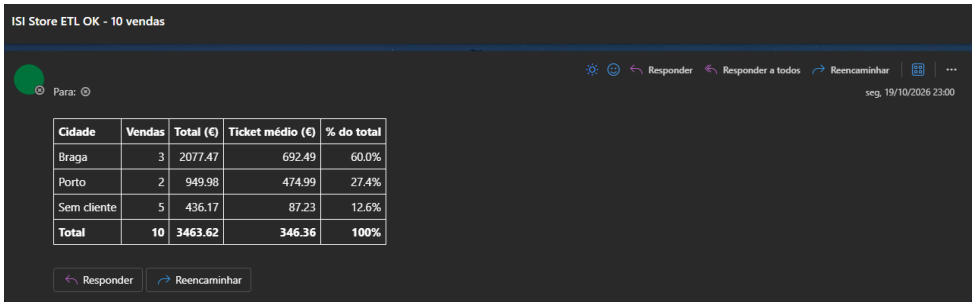


Figura 21: Correio eletrónico de alerta com totais por cidade.

4.7 Persistência e exportação

Antes da carga, confirmou-se a criação das tabelas e a ligação à base de dados. A Figura 22 mostra as tabelas `dw_vendas` e `stock`; a Figura 23 o conteúdo de *stock*; a Figura 24 a tabela `dw_vendas` vazia antes da carga; e a Figura 25 a ligação PostgreSQL testada com sucesso.

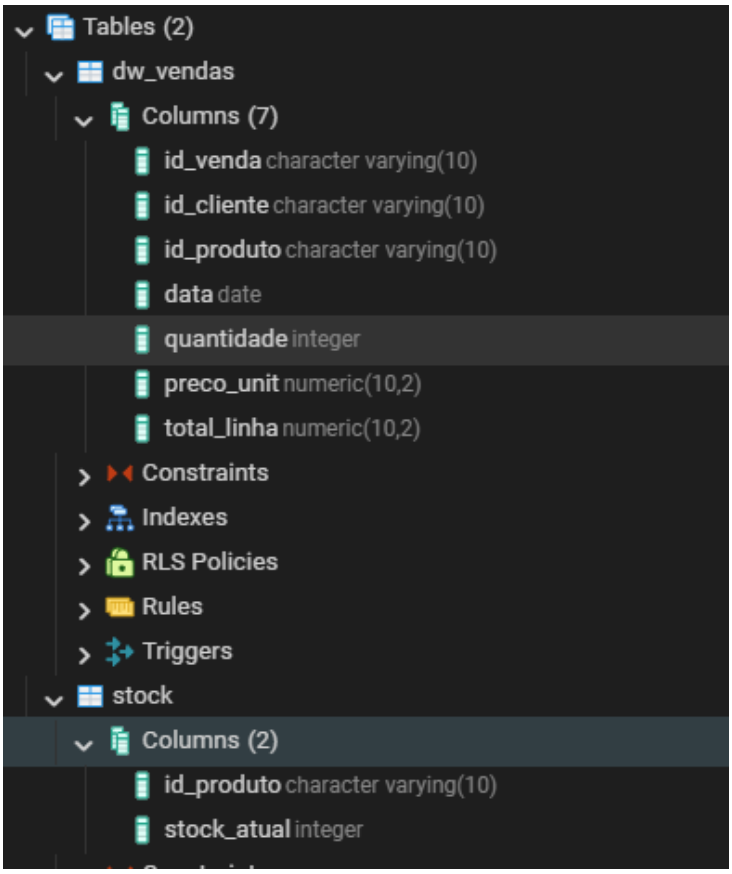


Figura 22: Tabelas `dw_vendas` e `stock` na base de dados `isi_store`.

	id_produto [PK] character varying (10)	stock_atual integer
1	P001	10
2	P002	3
3	P003	50
4	P004	2
5	P005	8
6	P006	15
7	P007	4
8	P008	6
9	P009	16
10	P010	58

Figura 23: Conteúdo da tabela `stock`.

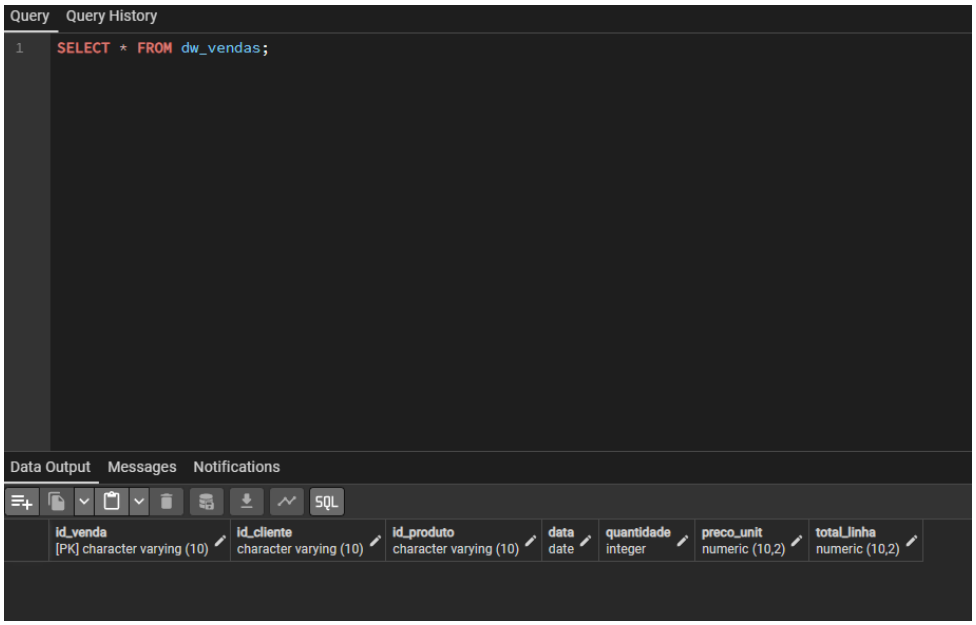


Figura 24: Tabela `dw_vendas` vazia antes da carga.

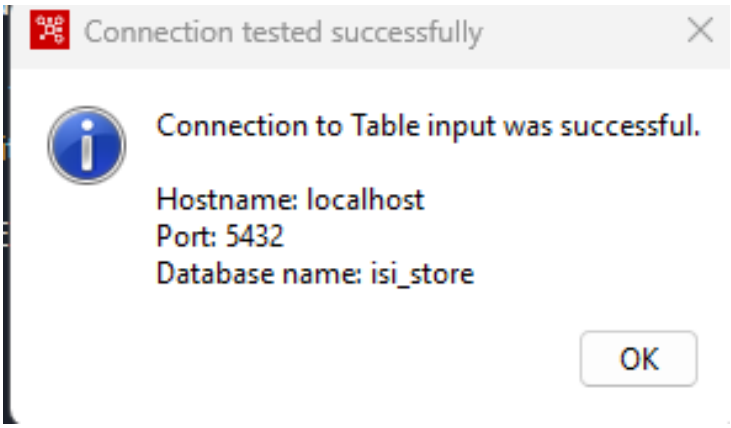


Figura 25: Ligação PostgreSQL `isi_store` testada com sucesso.

A carga persistiu 10 linhas em `dw_vendas` (Figura 26) e gerou `vendas_limpas.csv`, com delimitador `;`, e `resumo.json` (Figura 27), ambos transferidos para a pasta `FTP_remoto`, que simula o servidor remoto. A transformação de conversão gerou ainda `produtos.json` em `Dados_Saida`.

Query Query History

```
1  SELECT id_venda, quantidade, preco_unit, total_linha
2  FROM dw_vendas ORDER BY id_venda;
```

Data Output Messages Notifications

SQL

	id_venda [PK] character varying (10)	quantidade integer	preco_unit numeric (10,2)	total_linha numeric (10,2)
1	V001	2	899.99	1799.98
2	V002	1	29.90	29.90
3	V003	1	899.99	899.99
4	V004	3	15.50	46.50
5	V006	5	15.50	77.50
6	V008	2	49.99	99.98
7	V009	1	49.99	49.99
8	V010	1	199.99	199.99
9	V011	2	79.90	159.80
10	V012	1	99.99	99.99

Figura 26: Tabela `dw_vendas` com 10 linhas e totais corretos (V001 1799.98).



Figura 27: Ficheiros exportados em `Dados_Saida`.

4.8 Node-RED e painel

Como ferramenta complementar, configurou-se o *broker* Mosquitto em `localhost:1883` [**?**]. O fluxo de *streaming* republica em ciclo as vendas reais de `vendas_limpas.csv` no tópico `isi/vendas` (Figura 28), e o painel lê o mesmo CSV para calcular indicadores ETL, receita e ticket médio por cidade, principais produtos, tabela por cidade e tabela de vendas em *streaming* MQTT (Figura 29), disponível em `.../ui` [**?**].

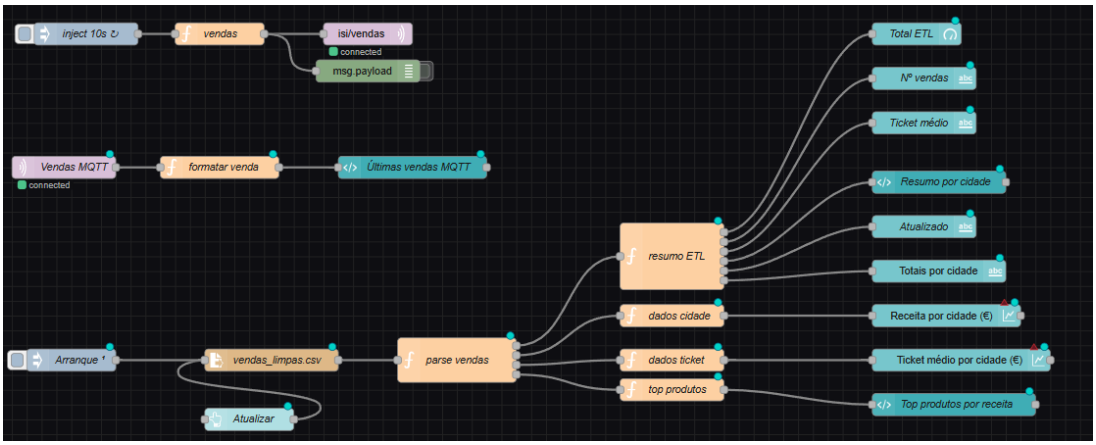


Figura 28: Fluxos Node-RED – *streaming* isi/vendas, agregação e painel.

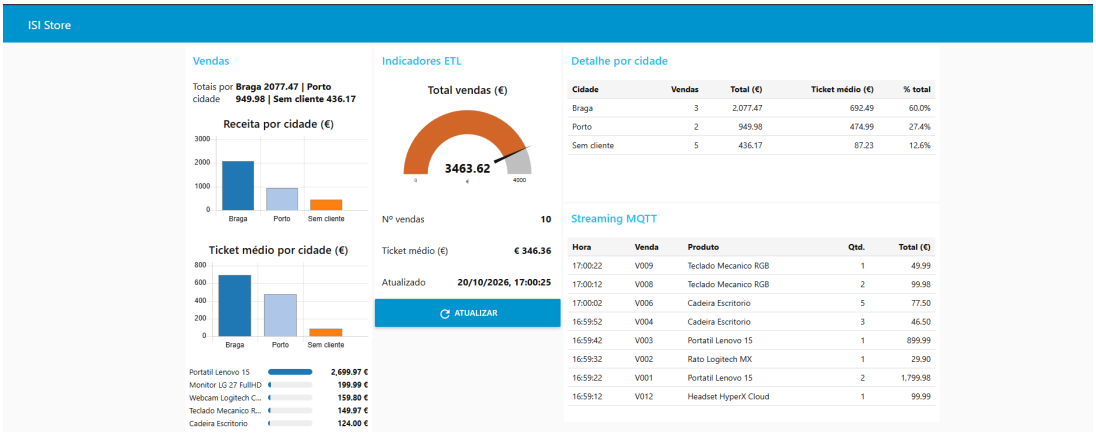


Figura 29: Painel calculado a partir de `vendas_limpas.csv`: indicadores ETL (total 3463.62), receita e ticket médio por cidade, principais produtos, tabela por cidade e *streaming* MQTT.

5 Análise e Discussão de Resultados

O presente capítulo analisa os resultados da execução e discute o cumprimento dos objetivos. A Secção 5.1 apresenta a validação quantitativa; a Secção 5.2 discute a qualidade, a integração e as limitações; e a Secção 5.3 documenta o vídeo de demonstração.

5.1 Validação da execução

A Tabela 8 sintetiza as contagens obtidas e as evidências associadas. A execução integral do *Job* terminou com sucesso e produziu os ficheiros e os registos esperados.

Etapa	Entrada	Saída	Evidência
Extração	13 vendas, 6 clientes, 158 produtos, 1 REST e 158 *stock*	5 fontes lidas	Figura 3
Limpeza de vendas	13 linhas	10 linhas	Figura 9
Validação de clientes	6 registos	2 válidos	Figura 13
Junção e agregação	10 vendas	3 grupos por cidade	Figura 18
Carga	10 vendas	10 linhas em `dw_vendas`	Figura 26
Exportação	10 vendas	CSV e JSON	Figura 27
Conversão	158 produtos	`produtos.json`	Figura 19
Orquestração	*Job* completo	Sucesso, correio e FTP	Figuras 20 e 21

Tabela 8: Síntese das contagens e evidências da execução.

A Tabela 8 confirma que o processo cumpriu as três fases obrigatórias: extração multiformato, transformação com limpeza e agregação, e carga com orquestração. Os totais por cidade foram Braga 2077.47/3, Porto 949.98/2 e sem cliente 436.17/5, num total de 3463.62 e ticket médio de 346.36.

5.2 Discussão dos resultados

A discussão organiza-se em três dimensões: qualidade e integridade dos dados, integração e orquestração, e limitações do trabalho.

5.2.1 Qualidade e integridade dos dados

A limpeza reduziu 13 vendas a 10. O duplicado V002 foi eliminado por Sort Rows e Unique Rows; V005 foi rejeitada por quantidade zero; V007 foi rejeitada por cliente nulo; e V009 foi corrigida pela substituição de `/` por `-`. A validação de clientes reduziu 6 registos a 2, após verificação de correio eletrónico, telemóvel e NIF.

A existência de vendas sem cliente deve ser interpretada como consequência da integridade referencial: os clientes inválidos foram excluídos, mas as vendas associadas permaneceram para não perder receita. O grupo sem cliente, com 436.17 e 5 vendas, evidencia a importância de distinguir dados ausentes de dados inválidos e de documentar essa decisão.

5.2.2 Integração e orquestração

O *Job* encadeou as transformações e as tarefas complementares sem intervenção manual. O correio eletrónico comunicou os totais por cidade, o FTP transferiu os ficheiros e o *log* registou o sucesso. Esta orquestração reduz o esforço operacional e torna a execução repetível.

O painel Node-RED complementa a integração *batch* com uma leitura do CSV consolidado e uma tabela de *streaming* MQTT. A atualização periódica permite observar vendas recentes, mas não substitui a fonte de verdade: o painel depende do ficheiro produzido pelo ETL.

5.2.3 Limitações e ameaças à validade

O FTP foi configurado num servidor local, devido à ausência de DNS externo no laboratório, pelo que a transferência não representa um ambiente de produção. Os dados são sintéticos e a execução foi validada numa única execução integral, o que limita a generalização dos resultados.

O painel não implementa autenticação nem controlo de acessos, e a dependência de `vendas_limpas.csv` introduz uma etapa manual entre o ETL e a atualização dos indicadores. O ficheiro `produtos.json` foi gerado localmente e não foi incluído na transferência FTP. Estas limitações não invalidam o processo, mas delimitam o seu âmbito.

5.3 Vídeo de demonstração

O vídeo com 3 a 5 minutos demonstra a execução do *Job*, os dados de entrada sujos, os resultados limpos, o correio eletrónico e o painel. Está disponível em `A-PREENCHER-LINK-YOUTUBE` (não listado).

A-PREENCHER-QR-CODE

6 Conclusão

O presente capítulo sintetiza o trabalho, identifica as contribuições, reconhece as limitações e propõe trabalhos futuros. A Secção 6.1 apresenta a síntese e as contribuições; a Secção 6.2 as limitações; e a Secção 6.3 os trabalhos futuros.

6.1 Síntese e contribuições

O trabalho cumpriu as três fases obrigatórias do enunciado [**?**]: extração multiformato validada por contagens, transformação com limpeza, expressões regulares, anonimização, junções e agregação, e carga com orquestração completa, incluindo correio eletrónico, FTP e registos. A execução produziu 10 vendas limpas, 2 clientes válidos, 158 produtos integrados e 10 linhas no *Data Mart*.

As contribuições principais são: (i) um processo ETL reprodutível para a ISI Store; (ii) uma especificação de requisitos e uma arquitetura lógica; (iii) regras de qualidade e anonimização alinhadas com o RGPD; (iv) a integração de orquestração, notificação e visualização; e (v) documentação que permite repetir e avaliar o processo. A comparação de plataformas e a discussão dos resultados constituem a componente de investigação aplicada.

6.2 Limitações

As limitações incluem o FTP local, a natureza sintética dos dados, a dependência do painel em relação ao CSV, a ausência de autenticação no painel e a inexistência de um agendador de produção. A validação numa única execução integral também limita a avaliação de desempenho e de robustez.

6.3 Trabalhos futuros

Como trabalhos futuros propõem-se a persistência em MongoDB, a visualização em Grafana em tempo real, a orquestração com Apache Airflow, a inclusão de testes automáticos de qualidade, a autenticação do painel e a integração do `produtos.json` no fluxo FTP. Propõe-se ainda a comparação com uma arquitetura de *streaming* baseada em Kafka ou MQTT industrial.

A Contagens de validação

Registaram-se as seguintes contagens ao longo do processo: `vendas.csv` com 13 linhas, `clientes.json` com 6 objetos, `produtos.xml` com 158 produtos, tabela `stock` com 158 linhas, vendas limpas com 10 linhas, clientes válidos com 2 registos, `dw_vendas` com 10 linhas e Group By com 3 linhas.